

```

1 %% Telco Customer Churn – Decision Tree vs Naive Bayes
2 clear; clc; close all;
3 % Force white background for all figures
4 set(groot, 'defaultFigureColor', 'w');
5 set(groot, 'defaultAxesColor', 'w');
6 set(groot, 'defaultAxesXColor', 'k');
7 set(groot, 'defaultAxesYColor', 'k');
8 set(groot, 'defaultTextColor', 'k');
9 rng(12345);
10
11 %% FIGURE SAVING SETTINGS
12 saveFigs = true;
13 figDir = "Figures";
14 if saveFigs && ~exist(figDir,"dir")
15     mkdir(figDir);
16 end
17
18 %% Load data
19 fname = "Telco-Customer-Churn.csv";
20 tbl = readtable(fname);
21
22 % Remove ID
23 if any(strcmpi(tbl.Properties.VariableNames,"customerID"))
24     tbl.customerID = [];
25 end
26
27 % Convert TotalCharges
28 if any(strcmpi(tbl.Properties.VariableNames,"TotalCharges"))
29     if iscell(tbl.TotalCharges) || isstring(tbl.TotalCharges)
30         tmp = string(tbl.TotalCharges);
31         tmp = strtrim(tmp);
32         tmp(tmp=="") = "NaN";
33         tbl.TotalCharges = double(tmp);
34     end
35 end
36
37 % Convert target to categorical
38 tbl.Churn = categorical(tbl.Churn);
39
40 % Convert any text predictors to categorical
41 varNames = tbl.Properties.VariableNames;
42 for i = 1:numel(varNames)
43     if strcmp(varNames{i},"Churn"), continue; end
44     v = tbl.(varNames{i});
45     if iscell(v) || isstring(v)
46         tbl.(varNames{i}) = categorical(v);
47     end
48 end
49
50 % Remove missing rows
51 tbl = rmmissing(tbl);
52
53 %% Basic EDA: target distribution
54 f1 = figure;
55 bar(countcats(tbl.Churn));
56 xticklabels(categories(tbl.Churn));
57 xlabel("Churn"); ylabel("Count");
58 title("Churn Distribution");
59 if saveFigs
60     exportgraphics(f1, fullfile(figDir,"01_churn_distribution.png"), "Resolution", 300);
61 end

```

```

62
63 %% Feature engineering
64 if any(strcmpi(varNames,"tenure")) && any(strcmpi(varNames,"TotalCharges"))
65     tbl.AvgMonthlySpend = tbl.TotalCharges ./ max(tbl.tenure, 1);
66 end
67 if any(strcmpi(varNames,"Contract"))
68     tbl.isLongTermContract = ismember(tbl.Contract, categorical(["One year","Two
year"]));
69 end
70
71 %% Split train/test
72 cv = cvpartition(height(tbl), "HoldOut", 0.2);
73 trainTbl = tbl(training(cv), :);
74 testTbl = tbl(test(cv), :);
75
76 Ytr = trainTbl.Churn;
77 Yte = testTbl.Churn;
78
79 predictorNames = setdiff(tbl.Properties.VariableNames, "Churn");
80
81
82 %% =====
83 %% BASELINE MODEL (Majority Class) - TEST SET
84 %% =====
85 % Majority class learned from TRAINING set
86 majorityClass = mode(Ytr);
87
88 % Predict majority class for all TEST samples
89 baselinePred = repmat(majorityClass, size(Yte));
90
91 % Baseline accuracy
92 baselineAcc = mean(baselinePred == Yte);
93
94 fprintf("Baseline (predict all '%s') Test Accuracy = %.3f\n", ...
95     string(majorityClass), baselineAcc);
96
97 % Confusion matrix for baseline
98 fBase = figure;
99 confusionchart(Yte, baselinePred);
100 title("Baseline (Majority Class) - Confusion Matrix (Test Set)");
101
102 if saveFigs
103     exportgraphics(fBase, fullfile(figDir,"00_baseline_confusion_test.png"), ...
104         "Resolution", 300);
105 end
106
107
108 %% ---- Encoding: dummy variables for categoricals ----
109 [Xtr, featNames] = designMatrixFromTable(trainTbl(:, predictorNames));
110 [Xte, ~] = designMatrixFromTable(testTbl(:, predictorNames), featNames);
111
112 %% =====
113 %% PART A) 5-FOLD CROSS-VALIDATION ON TRAINING SET
114 %% =====
115 K = 5;
116 cvK = cvpartition(Ytr, "Kfold", K);
117
118 accNB = zeros(K,1);
119 accTR = zeros(K,1);
120 aucNB = zeros(K,1);
121 aucTR = zeros(K,1);

```

```

122
123 for k = 1:K
124     idxTrain = training(cvK, k);
125     idxVal    = test(cvK, k);
126
127     Xtr_k = Xtr(idxTrain,:);
128     Ytr_k = Ytr(idxTrain);
129     Xval_k = Xtr(idxVal,:);
130     Yval_k = Ytr(idxVal);
131
132     % Standardize INSIDE each fold for Naive Bayes (prevents leakage)
133     [XtrZ_k, mu_k, sig_k] = zscore(Xtr_k);
134     sig_k(sig_k==0) = 1;
135     XvalZ_k = (Xval_k - mu_k) ./ sig_k;
136
137     % --- Naive Bayes ---
138     nb_k = fitcnb(XtrZ_k, Ytr_k);
139     [predNB_k, scoreNB_k] = predict(nb_k, XvalZ_k);
140     accNB(k) = mean(predNB_k == Yval_k);
141     [~,~,~,aucNB(k)] = perfcurve(Yval_k, scoreNB_k(:,2), "Yes");
142
143     % --- Decision Tree ---
144     tree_k = fitctree(Xtr_k, Ytr_k, "MinLeafSize", 10);
145     [predTR_k, scoreTR_k] = predict(tree_k, Xval_k);
146     accTR(k) = mean(predTR_k == Yval_k);
147     [~,~,~,aucTR(k)] = perfcurve(Yval_k, scoreTR_k(:,2), "Yes");
148 end
149
150 cvResults = table( ...
151     ["Naive Bayes"; "Decision Tree"], ...
152     [mean(accNB); mean(accTR)], ...
153     [std(accNB); std(accTR)], ...
154     [mean(aucNB); mean(aucTR)], ...
155     [std(aucNB); std(aucTR)], ...
156     'VariableNames', {'Model', 'MeanAccuracy', 'StdAccuracy', 'MeanAUC', 'StdAUC'});
157
158 disp("=== 5-Fold Cross-Validation Results (on training set) ===");
159 disp(cvResults);
160
161 if saveFigs
162     writetable(cvResults, fullfile(figDir,"02_cv_results.csv"));
163 end
164
165 % Plot per-fold accuracies
166 fCV = figure;
167 bar([accNB accTR]);
168 xlabel("Fold");
169 ylabel("Accuracy");
170 title("5-Fold Cross-Validation Accuracy per Fold");
171 legend("Naive Bayes","Decision Tree","Location","best");
172 grid on;
173
174 if saveFigs
175     exportgraphics(fCV, fullfile(figDir,"03_cv_accuracy_per_fold.png"), "Resolution",
176     300);
177 end
178
179
180 %% =====
181 %% HYPERPARAMETER TUNING (Decision Tree) using CV on TRAINING SET

```

```

182 %% =====
183 % Small, coursework-friendly grid search (do NOT overdo it)
184 leafGrid = [1 2 5 10 20 50];
185 splitGrid = [10 25 50 100];
186
187 tuningRows = [];
188
189 for ls = leafGrid
190     for ms = splitGrid
191         % CV estimates for this hyperparameter combo
192         accFold = zeros(K,1);
193         aucFold = zeros(K,1);
194
195         for k = 1:K
196             idxTrain = training(cvK, k);
197             idxVal = test(cvK, k);
198
199             Xtr_k = Xtr(idxTrain,:);
200             Ytr_k = Ytr(idxTrain);
201             Xval_k = Xtr(idxVal,:);
202             Yval_k = Ytr(idxVal);
203
204             tree_k = fitctree(Xtr_k, Ytr_k, "MinLeafSize", ls, "MaxNumSplits", ms);
205             [pred_k, score_k] = predict(tree_k, Xval_k);
206
207             accFold(k) = mean(pred_k == Yval_k);
208             [~,~,~,aucFold(k)] = perfcurve(Yval_k, score_k(:,2), "Yes");
209         end
210
211         tuningRows = [tuningRows; ...
212             ls, ms, mean(accFold), std(accFold), mean(aucFold), std(aucFold)];
213     end
214 end
215
216 tuningResults = array2table(tuningRows, ...
217     'VariableNames',
218     {'MinLeafSize', 'MaxNumSplits', 'MeanAcc', 'StdAcc', 'MeanAUC', 'StdAUC'});
219
220 % Pick the best model by MeanAUC first, then MeanAcc
221 tuningResults = sortrows(tuningResults, {'MeanAUC', 'MeanAcc'}, {'descend', 'descend'});
222 bestParams = tuningResults(1,:);
223
224 disp("=== Decision Tree Hyperparameter Tuning Results (Top 10) ===");
225 disp(tuningResults(1:min(10,height(tuningResults)),:));
226
227 disp("=== Best Decision Tree Hyperparameters (by CV MeanAUC) ===");
228 disp(bestParams);
229
230 if saveFigs
231     writetable(tuningResults,
232         fullfile(figDir,"09_tree_hyperparameter_tuning_results.csv"));
233 end
234
235 % Plot: MeanAUC as a heatmap-like matrix (MaxNumSplits x MinLeafSize)
236 aucMat = NaN(numel(splitGrid), numel(leafGrid));
237 for i = 1:height(tuningResults)
238     ls = tuningResults.MinLeafSize(i);
239     ms = tuningResults.MaxNumSplits(i);
240     r = find(splitGrid==ms);
241     c = find(leafGrid==ls);
242     aucMat(r,c) = tuningResults.MeanAUC(i);

```

```

241 end
242
243 fTune = figure;
244 imagesc(aucMat);
245 colorbar;
246 title("Decision Tree Tuning (CV Mean AUC)");
247 xlabel("MinLeafSize");
248 ylabel("MaxNumSplits");
249 xticks(1:numel(leafGrid)); xticklabels(string(leafGrid));
250 yticks(1:numel(splitGrid)); yticklabels(string(splitGrid));
251
252 if saveFigs
253     exportgraphics(fTune, fullfile(figDir,"10_tree_tuning_heatmap_auc.png"),
254         "Resolution", 300);
255 end
256
257
258
259 %% =====
260 %% FINAL TRAINING ON FULL TRAIN SET + TEST SET EVALUATION
261 %% =====
262
263 %% Standardization (Naive Bayes) using TRAINING stats only
264 [XtrZ, mu, sig] = zscore(Xtr);
265 sig(sig==0) = 1;
266 XteZ = (Xte - mu) ./ sig;
267
268 %% Model 1: Naive Bayes (Gaussian NB) - final
269 nb = fitcnb(XtrZ, Ytr);
270 [nbPred, nbScore] = predict(nb, XteZ);
271
272 %% Model 2: Decision Tree - final
273 % Use tuned hyperparameters from CV
274 tree = fitctree(Xtr, Ytr, ...
275     "MinLeafSize", bestParams.MinLeafSize, ...
276     "MaxNumSplits", bestParams.MaxNumSplits);
277 [treePred, treeScore] = predict(tree, Xte);
278
279 %% Final evaluation (prints metrics)
280 disp("=== FINAL TEST SET RESULTS ===");
281
282 disp("Naive Bayes:");
283 showMetrics(Yte, nbPred, nbScore, "Naive Bayes (Test)");
284
285 disp("Decision Tree:");
286 showMetrics(Yte, treePred, treeScore, "Decision Tree (Test)");
287
288 %% Confusion Matrices
289 f2 = figure;
290 confusionchart(Yte, nbPred);
291 title("Naive Bayes - Confusion Matrix (Test Set)");
292 if saveFigs
293     exportgraphics(f2, fullfile(figDir,"04_confusion_naive_bayes_test.png"),
294         "Resolution", 300);
295 end
296
297 f3 = figure;
298 confusionchart(Yte, treePred);
299 title("Decision Tree - Confusion Matrix (Test Set)");
300 if saveFigs

```

```

300     exportgraphics(f3, fullfile(figDir,"05_confusion_decision_tree_test.png"),
"Resolution", 300);
301 end
302
303 %% ROC curves
304 posClass = "Yes";
305 [XB, YB, ~, AUC_nb] = perfcurve(Yte, nbScore(:,2), posClass);
306 [XT, YT, ~, AUC_tr] = perfcurve(Yte, treeScore(:,2), posClass);
307
308 f4 = figure;
309 plot(XB, YB, "-"); hold on;
310 plot(XT, YT, "--");
311 xlabel("False Positive Rate"); ylabel("True Positive Rate");
312 title("ROC Curves (Test Set)");
313 legend("Naive Bayes (AUC=" + string(round(AUC_nb,3)) + ")", ...
"Decision Tree (AUC=" + string(round(AUC_tr,3)) + ")", ...
"Location","SouthEast");
316 grid on;
317
318 if saveFigs
319     exportgraphics(f4, fullfile(figDir,"06_roc_curves_test.png"), "Resolution", 300);
320 end
321
322
323
324 %% Precision-Recall Curves (Test Set)
325 [posX_nb, posY_nb, ~, AUPRC_nb] = perfcurve(Yte, nbScore(:,2), "Yes", "xCrit","reca",
"yCrit","prec");
326 [posX_tr, posY_tr, ~, AUPRC_tr] = perfcurve(Yte, treeScore(:,2), "Yes", "xCrit","reca",
"yCrit","prec");
327
328 fPR = figure;
329 plot(posX_nb, posY_nb, "-"); hold on;
330 plot(posX_tr, posY_tr, "--");
331 xlabel("Recall"); ylabel("Precision");
332 title("Precision-Recall Curves (Test Set)");
333 legend("Naive Bayes (AUPRC=" + string(round(AUPRC_nb,3)) + ")", ...
"Decision Tree (AUPRC=" + string(round(AUPRC_tr,3)) + ")", ...
"Location","best");
336 grid on;
337
338 if saveFigs
339     exportgraphics(fPR, fullfile(figDir,"13_precision_recall_curves_test.png"),
"Resolution", 300);
340 end
341
342
343 %% =====
344 %% FINAL COMPARISON TABLE: CV vs TEST (Naive Bayes vs Tuned Decision Tree)
345 %% =====
346 % Compute TEST metrics into tables
347 testNB = computeMetricsTable(Yte, nbPred, nbScore, "Naive Bayes", AUC_nb);
348 testTR = computeMetricsTable(Yte, treePred, treeScore, "Decision Tree (Tuned)", AUC_tr);
349
350 % Pull CV means/stds for NB and Tree(Tuned)
351 % NB CV stats from earlier (accNB/aucNB arrays)
352 cvNB_meanAcc = mean(accNB); cvNB_stdAcc = std(accNB);
353 cvNB_meanAUC = mean(aucNB); cvNB_stdAUC = std(aucNB);
354
355 % Tree tuned CV stats from tuningResults/bestParams
356 cvTR_meanAcc = bestParams.MeanAcc;

```

```

357 cvTR_stdAcc = bestParams.StdAcc;
358 cvTR_meanAUC = bestParams.MeanAUC;
359 cvTR_stdAUC = bestParams.StdAUC;
360
361 finalComparison = table( ...
362     ["Naive Bayes"; "Decision Tree (Tuned)"], ...
363     [cvNB_meanAcc; cvTR_meanAcc], ...
364     [cvNB_stdAcc; cvTR_stdAcc], ...
365     [cvNB_meanAUC; cvTR_meanAUC], ...
366     [cvNB_stdAUC; cvTR_stdAUC], ...
367     [testNB.Accuracy; testTR.Accuracy], ...
368     [testNB.F1; testTR.F1], ...
369     [testNB.AUC; testTR.AUC], ...
370     'VariableNames',
371     {'Model', 'CV_MeanAcc', 'CV_StdAcc', 'CV_MeanAUC', 'CV_StdAUC', 'Test_Accuracy', 'Test_F1', 'Test_AUC'});
372 disp("=== FINAL COMPARISON (CV vs TEST) ===");
373 disp(finalComparison);
374
375 if saveFigs
376     writetable(finalComparison, fullfile(figDir, "11_final_comparison_cv_vs_test.csv"));
377 end
378
379 % Optional plot: compare CV mean AUC vs Test AUC
380 fComp = figure;
381 bar([finalComparison.CV_MeanAUC finalComparison.Test_AUC]);
382 xticklabels(finalComparison.Model);
383 ylabel("AUC");
384 title("AUC Comparison: CV Mean vs Test");
385 legend("CV Mean AUC", "Test AUC", "Location", "best");
386 grid on;
387
388 if saveFigs
389     exportgraphics(fComp, fullfile(figDir, "12_auc_comparison_cv_vs_test.png"),
390         "Resolution", 300);
391 end
392
393 %% Feature importance (Decision Tree)
394 imp = predictorImportance(tree);
395 [impS, idx] = sort(imp, "descend");
396 topK = min(20, numel(impS));
397
398 f5 = figure;
399 bar(impS(1:topK));
400 title("Decision Tree Feature Importance (Top 20) - Test Run");
401 xticks(1:topK);
402 xticklabels(featNames(idx(1:topK)));
403 xtickangle(45);
404 ylabel("Importance");
405
406 if saveFigs
407     exportgraphics(f5, fullfile(figDir, "07_tree_feature_importance_top20.png"),
408         "Resolution", 300);
409 end
410
411 %% Save a text summary of the decision tree model
412 if saveFigs
413     txtFile = fullfile(figDir, "08_tree_model_summary.txt");
414     fid = fopen(txtFile, "w");

```

```

414     fprintf(fid, "Decision Tree Model Summary\n\n");
415     fprintf(fid, "%s\n", evalc("disp(tree)"));
416     fclose(fid);
417 end
418
419 disp("DONE.");
420
421 %% ===== Helper functions =====
422 function [X, featNames] = designMatrixFromTable(T, featNamesRef)
423 % Creates numeric matrix from a table:
424 % numeric columns -> append
425 % categorical columns -> dummyvar
426 % If featNamesRef provided, aligns columns to reference (for test set).
427
428 vars = T.Properties.VariableNames;
429 X = [];
430 featNames = strings(1,0);
431
432 for i = 1:numel(vars)
433     vname = vars{i};
434     col = T.(vname);
435
436     if isnumeric(col) || islogical(col)
437         X = [X, double(col)];
438         featNames = [featNames, string(vname)];
439     else
440         if ~iscategorical(col)
441             col = categorical(col);
442         end
443         cats = categories(col);
444         D = dummyvar(col);
445         X = [X, D];
446         featNames = [featNames, strcat(string(vname), "_", string(cats'))];
447     end
448 end
449
450 if nargin == 2
451     ref = featNamesRef;
452     Xaligned = zeros(height(T), numel(ref));
453     [lia, locb] = ismember(featNames, ref);
454     Xaligned(:, locb(lia)) = X(:, lia);
455     X = Xaligned;
456     featNames = ref;
457 end
458 end
459
460 function showMetrics(Ytrue, Ypred, scores, modelName)
461 % Prints Accuracy, Precision, Recall, F1, AUC for positive class "Yes"
462 posClass = "Yes";
463
464 tp = sum((Ypred==posClass) & (Ytrue==posClass));
465 tn = sum((Ypred~=posClass) & (Ytrue~=posClass));
466 fp = sum((Ypred==posClass) & (Ytrue~=posClass));
467 fn = sum((Ypred~=posClass) & (Ytrue==posClass));
468
469 acc = (tp+tn) / max(1,(tp+tn+fp+fn));
470 prec = tp / max(1,(tp+fp));
471 rec = tn / max(1,(tn+fn));
472 f1 = 2*(prec*rec) / max(eps,(prec+rec));
473
474 [~,~,~,auc] = perfcurve(Ytrue, scores(:,2), posClass);

```



```

475
476 fprintf("%s | Acc=%.3f  Prec=%.3f  Rec=%.3f  F1=%.3f  AUC=%.3f\n", ...
477         modelName, acc, prec, rec, f1, auc);
478 end
479
480
481
482 function out = computeMetricsTable(Ytrue, Ypred, scores, modelName, aucVal)
483 % Returns a 1-row table of key test metrics (same logic as showMetrics)
484
485 posClass = "Yes";
486
487 tp = sum((Ypred==posClass) & (Ytrue==posClass));
488 tn = sum((Ypred~=posClass) & (Ytrue~=posClass));
489 fp = sum((Ypred==posClass) & (Ytrue~=posClass));
490 fn = sum((Ypred~=posClass) & (Ytrue==posClass));
491
492 acc = (tp+tn) / max(1,(tp+tn+fp+fn));
493 prec = tp / max(1,(tp+fp));
494 rec = tp / max(1,(tp+fn));
495 f1 = 2*(prec*rec) / max(eps,(prec+rec));
496
497 out = table(string(modelName), acc, prec, rec, f1, aucVal, ...
498             'VariableNames', {'Model','Accuracy','Precision','Recall','F1','AUC'});
499 end
500

```